

Task: Exploratory Data Analysis (EDA) and Machine Learning on Agricultural Yield Dataset

Part A: Understanding the Dataset

Q1. Dataset Overview

Code:

```
import pandas as pd

df = pd.read_csv("agriculture_yield_dataset.csv")

print("Rows and Columns:")
print(df.shape)

print("\nColumn Names:")
print(df.columns)

print("\nFirst 10 Records:")
print(df.head(10))
```

Output:

```
*** Rows and Columns:
(1500, 8)

Column Names:
Index(['rainfall_mm', 'temperature_c', 'fertilizer_kg', 'irrigation_hours',
       'soil_ph', 'crop_type', 'soil_type', 'yield_ton_per_hectare'],
      dtype='object')

First 10 Records:
   rainfall_mm  temperature_c  fertilizer_kg  irrigation_hours  soil_ph  \
0      588.6         18.6         242.4           6.5         6.5
1      772.8         34.6         247.2          10.0         6.5
2      970.9         36.3         168.4           7.3         6.4
3      611.7         19.0         121.7           3.7         6.0
4      696.1         29.6         184.6           5.1         6.1
5      831.9         28.0         190.3           2.1         6.1
6     1023.8         32.0         108.5           6.9         6.2
7     1142.4         18.4         241.9           4.1         7.3
8      810.4         36.4         164.8           9.9         6.5
9     1085.5         29.4          89.3           8.3         5.8

   crop_type  soil_type  yield_ton_per_hectare
0     Maize     Loamy           5.92
1     Maize     Sandy           6.24
2     Wheat     Sandy           4.77
3     Maize     Loamy           4.04
4     Cotton    Clay           5.17
5     Soybean   Loamy           5.50
6     Cotton   Loamy           5.44
7     Soybean   Sandy           5.88
8     Wheat     Loamy           6.35
9     Cotton     Loamy           6.30
```

Q2. Data Types and Missing Values

Code:

```
print("Data Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
```

Output:

```
▶ print("Data Types:")
  print(df.dtypes)

  print("\nMissing Values:")
  print(df.isnull().sum())

*** Data Types:
rainfall_mm      float64
temperature_c    float64
fertilizer_kg     float64
irrigation_hours  float64
soil_ph          float64
crop_type        object
soil_type         object
yield_ton_per_hectare float64
dtype: object

Missing Values:
rainfall_mm      0
temperature_c    0
fertilizer_kg     0
irrigation_hours  0
soil_ph          0
crop_type        0
soil_type         0
yield_ton_per_hectare 0
dtype: int64
```

Q3. Descriptive Statistics

Code:

```
print(df.describe())

print("\nMean Values:")
print(df.mean(numeric_only=True))

print("\nStandard Deviation:")
print(df.std(numeric_only=True))
```

Output:

```
***
count    rainfall_mm    temperature_c    fertilizer_kg    irrigation_hours  \
mean      754.054667      27.749467      148.744067      5.403267
std       255.097216      5.758101      56.990279      2.584329
min       300.200000      18.000000      50.300000      1.000000
25%       536.175000      22.600000      98.600000      3.200000
50%       761.200000      27.700000      146.850000      5.400000
75%       964.375000      32.600000      196.575000      7.600000
max      1200.000000      38.000000      249.900000      10.000000

count    soil_ph    yield_ton_per_hectare
mean      6.759133      5.028793
std       0.719742      0.968282
min       5.500000      2.090000
25%       6.100000      4.337500
50%       6.800000      5.010000
75%       7.400000      5.740000
max       8.000000      7.860000

Mean Values:
rainfall_mm      754.054667
temperature_c    27.749467
fertilizer_kg    148.744067
irrigation_hours  5.403267
soil_ph          6.759133
yield_ton_per_hectare  5.028793
dtype: float64

Standard Deviation:
rainfall_mm      255.097216
temperature_c    5.758101
fertilizer_kg    56.990279
irrigation_hours  2.584329
soil_ph          0.719742
yield_ton_per_hectare  0.968282
dtype: float64
```

Code:

```
means = df.mean(numeric_only=True)

print(means.idxmax())
print(means.max())
```

Output:

```
means = df.mean(numeric_only=True)

print(means.idxmax())
print(means.max())

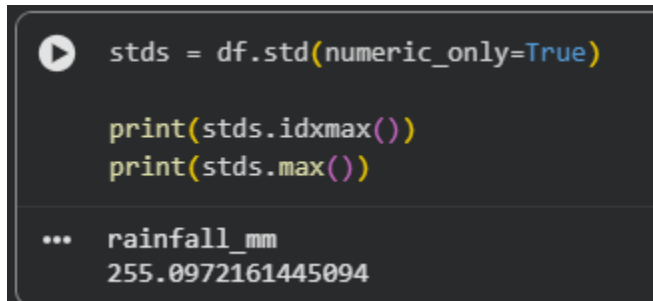
*** rainfall_mm
754.0546666666667
```

Code:

```
stds = df.std(numeric_only=True)

print(stds.idxmax())
print(stds.max())
```

Output:



```
stds = df.std(numeric_only=True)

print(stds.idxmax())
print(stds.max())

... rainfall_mm
255.0972161445094
```

Part B: Exploratory Data Analysis (EDA)

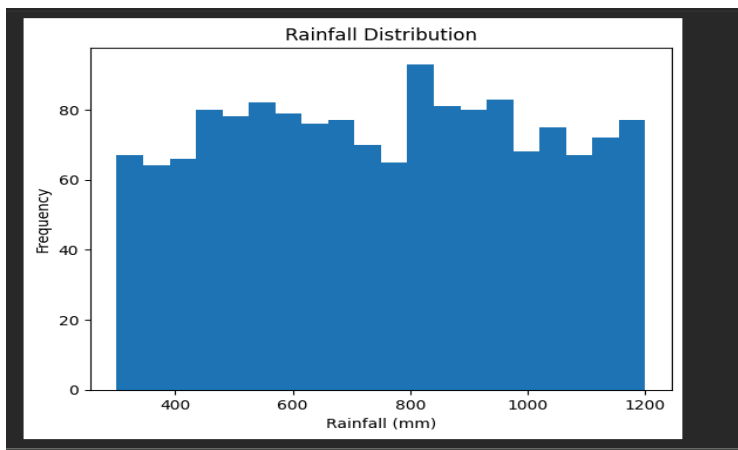
Q4. Distribution Analysis

Code:

```
import matplotlib.pyplot as plt

plt.hist(df["rainfall_mm"], bins=20)
plt.title("Rainfall Distribution")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Frequency")
plt.show()
```

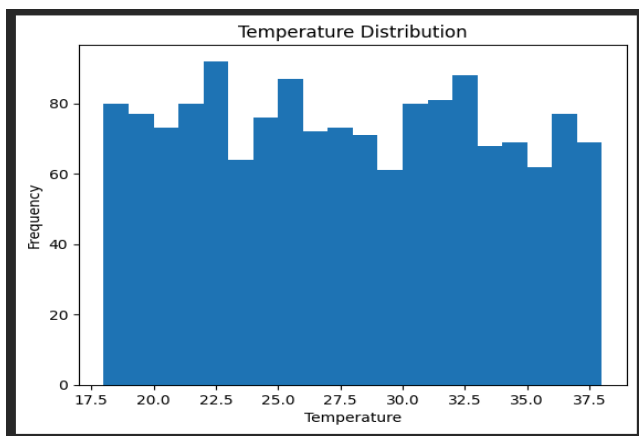
Output:



Code:

```
plt.hist(df["temperature_c"], bins=20)
plt.title("Temperature Distribution")
plt.xlabel("Temperature")
plt.ylabel("Frequency")
plt.show()
```

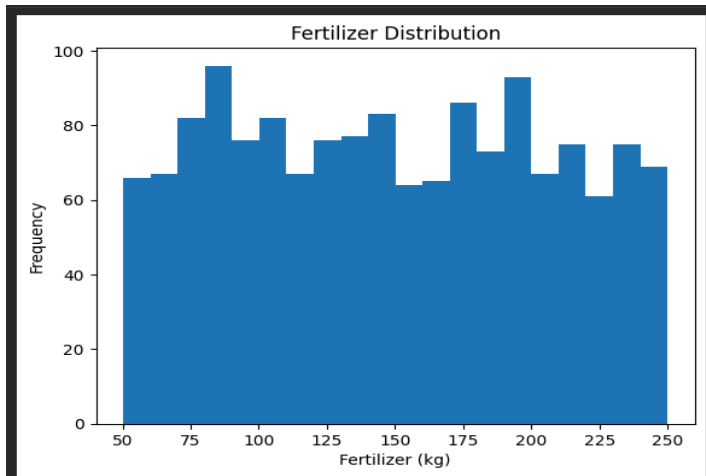
Output:



Code:

```
plt.hist(df["fertilizer_kg"], bins=20)
plt.title("Fertilizer Distribution")
plt.xlabel("Fertilizer (kg)")
plt.ylabel("Frequency")
plt.show()
```

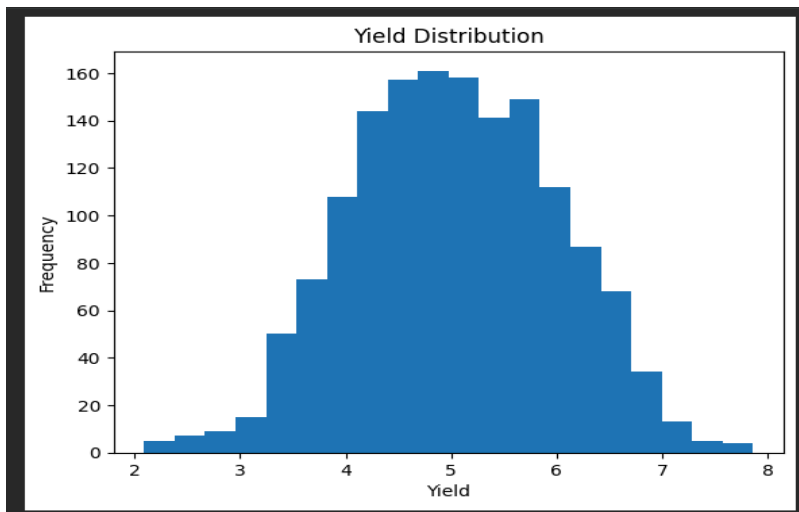
Output:



Code:

```
plt.hist(df["yield_ton_per_hectare"], bins=20)
plt.title("Yield Distribution")
plt.xlabel("Yield")
plt.ylabel("Frequency")
plt.show()
```

Output:



Q5. Crop Type Analysis

Code:

```
print(df["crop_type"].value_counts())
```

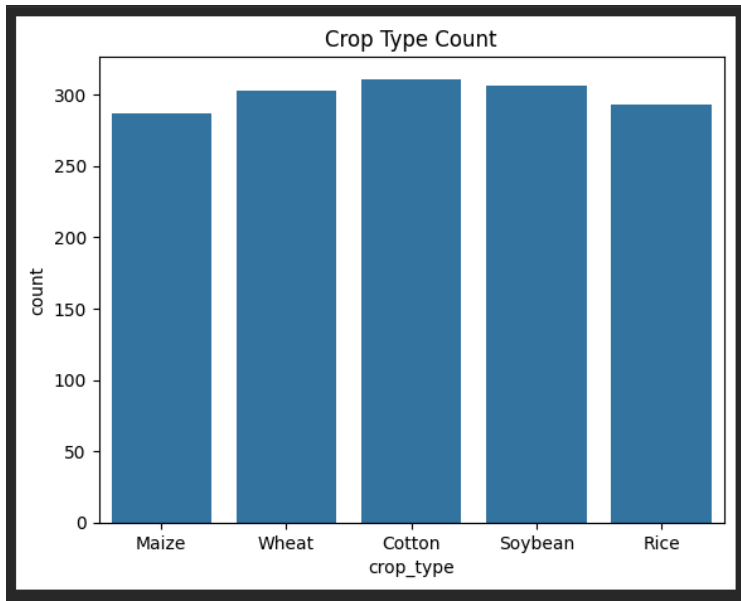
Output:

```
print(df["crop_type"].value_counts())  
  
... crop_type  
Cotton    311  
Soybean   306  
Wheat     303  
Rice      293  
Maize     287  
Name: count, dtype: int64
```

Code:

```
import seaborn as sns  
  
sns.countplot(x="crop_type", data=df)  
plt.title("Crop Type Count")  
plt.show()
```

Output:



Code:

```
print(df["crop_type"].value_counts().idxmax())
```

Output:

```
print(df["crop_type"].value_counts().idxmax())  
  
... Cotton
```

Q6. Soil Type Analysis

Code:

```
print(df["soil_type"].value_counts())
```

Output:

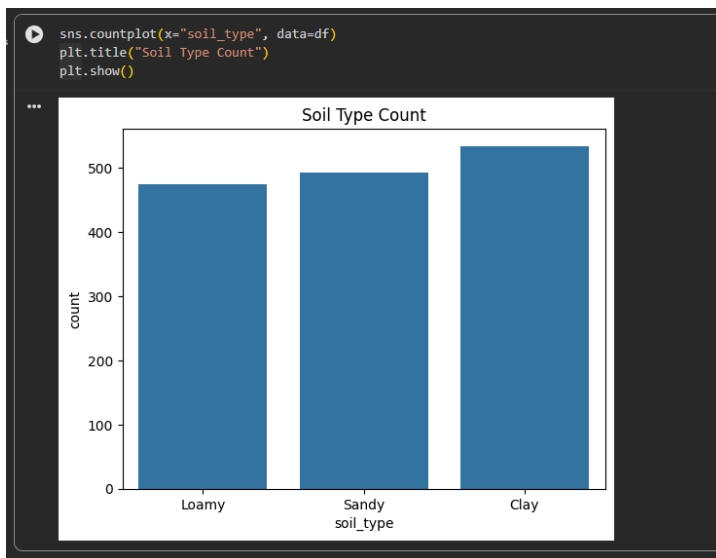
```
print(df["soil_type"].value_counts())

... soil_type
Clay      534
Sandy     492
Loamy     474
Name: count, dtype: int64
```

Code:

```
sns.countplot(x="soil_type", data=df)
plt.title("Soil Type Count")
plt.show()
```

Output:



Code:

```
print(df["soil_type"].value_counts().idxmax())
```

Output:

```
print(df["soil_type"].value_counts().idxmax())

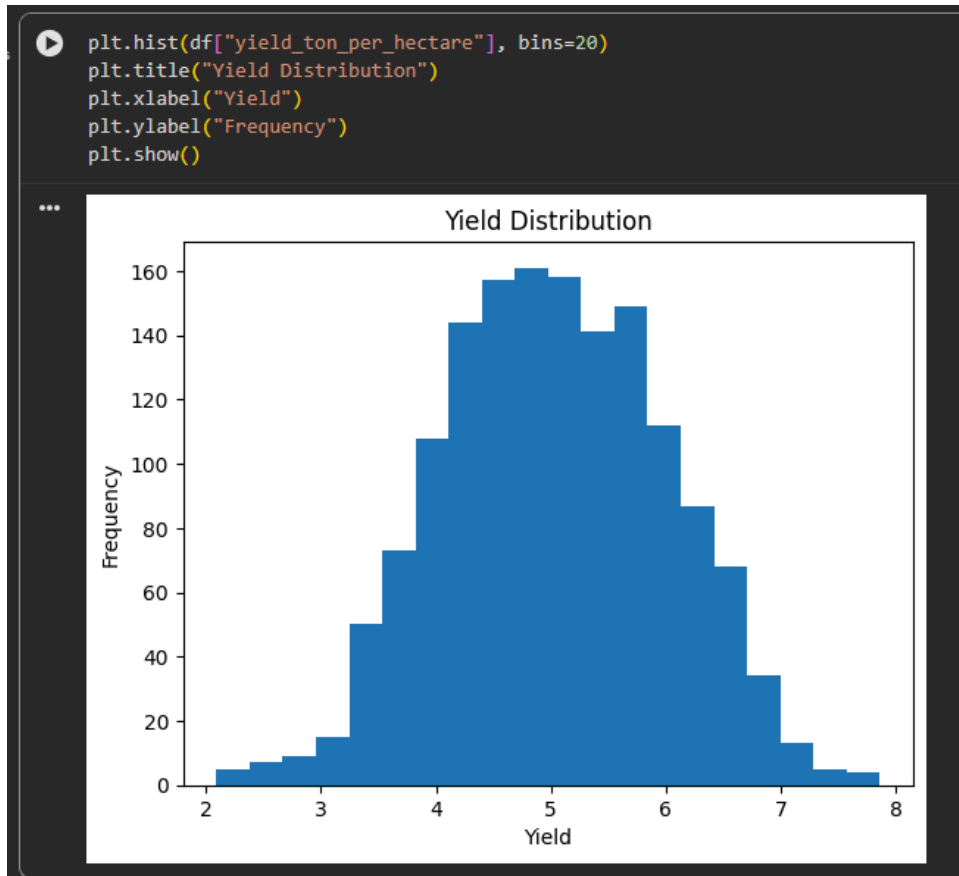
... Clay
```


Q7. Yield Distribution

Code:

```
plt.hist(df["yield_ton_per_hectare"], bins=20)
plt.title("Yield Distribution")
plt.xlabel("Yield")
plt.ylabel("Frequency")
plt.show()
```

Output:

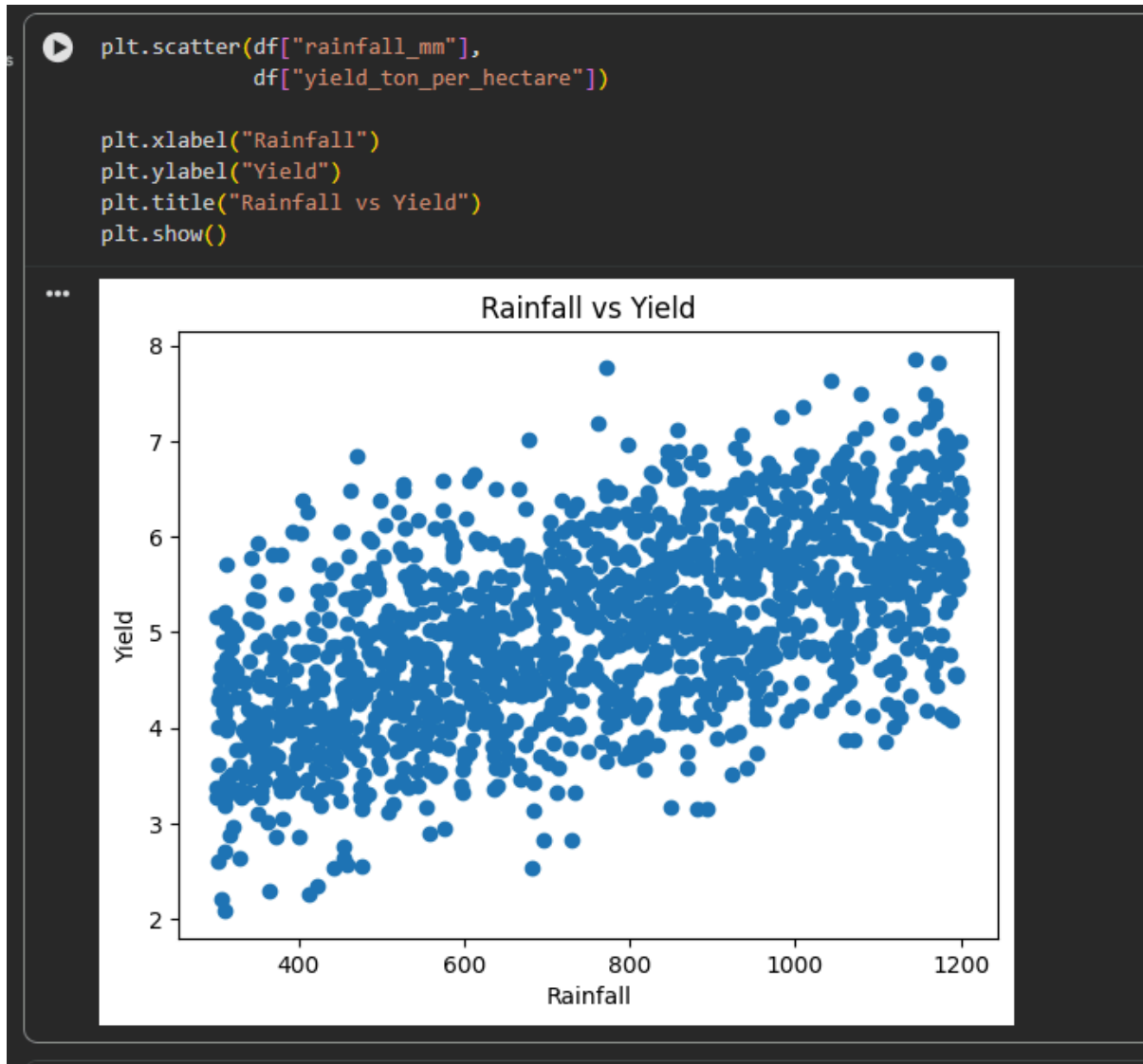


Q8. Scatter Plot Analysis

Code:

```
plt.scatter(df["rainfall_mm"],  
            df["yield_ton_per_hectare"])  
plt.xlabel("Rainfall")  
plt.ylabel("Yield")  
plt.title("Rainfall vs Yield")  
plt.show()
```

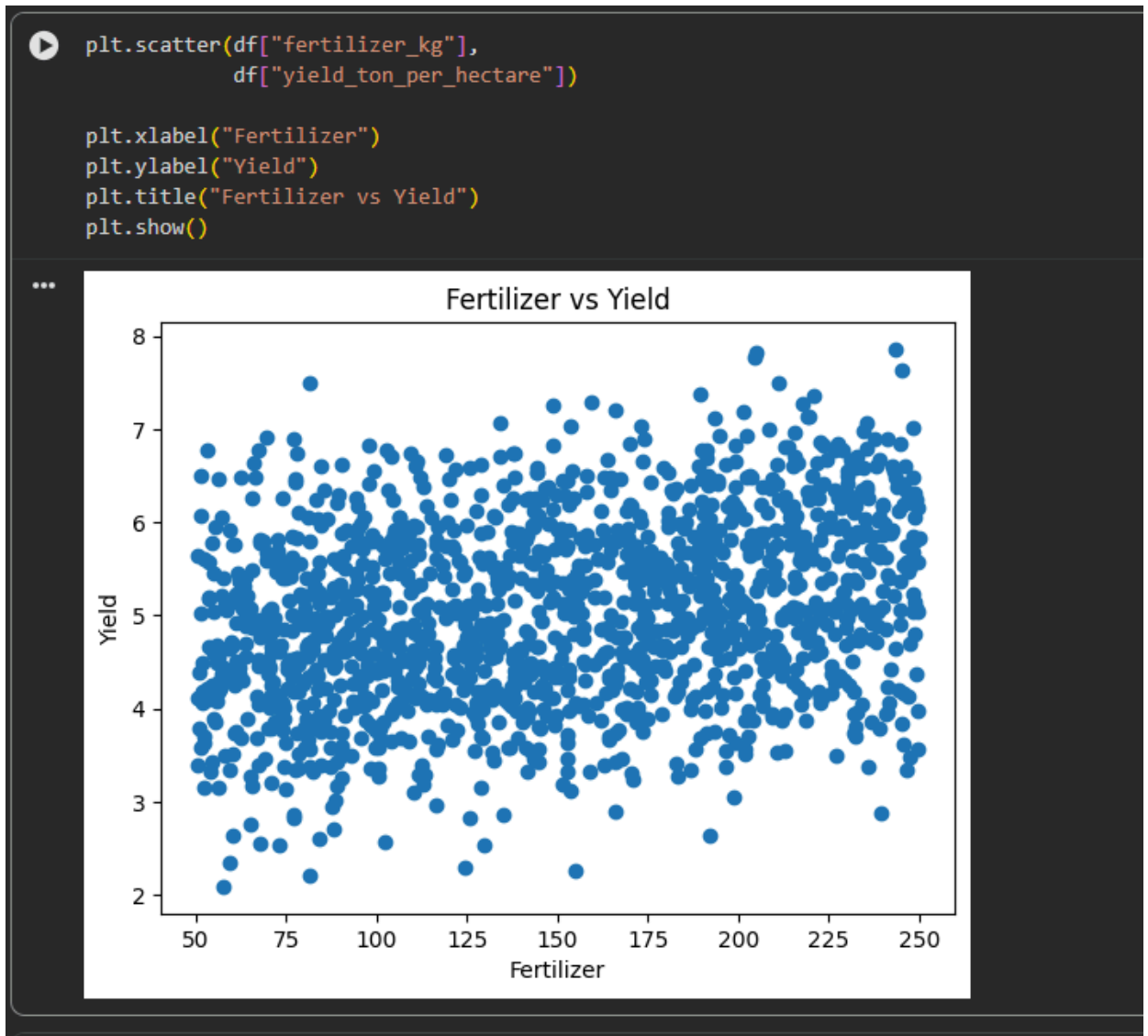
Output:



Code:

```
plt.scatter(df["fertilizer_kg"],
            df["yield_ton_per_hectare"])
plt.xlabel("Fertilizer")
plt.ylabel("Yield")
plt.title("Fertilizer vs Yield")
plt.show()
print(df.head(10))
```

Output:



Q9. Correlation Matrix

Code:

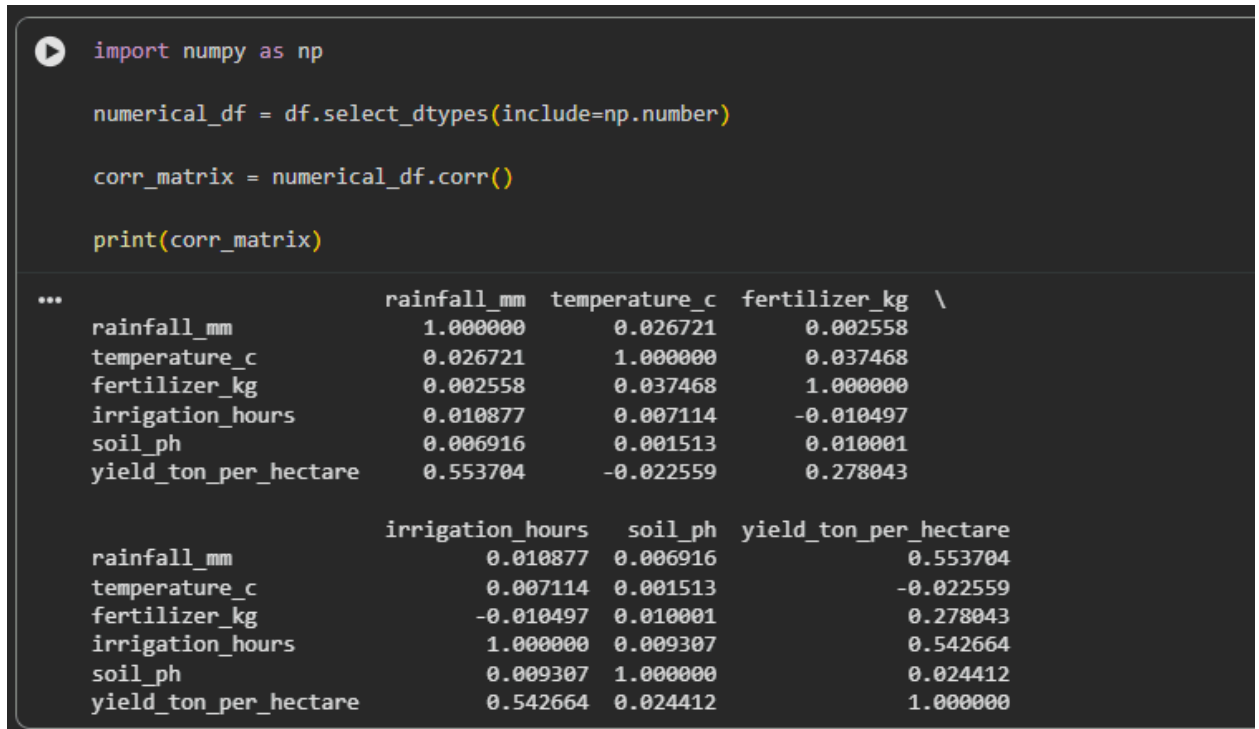
```
import numpy as np

numerical_df = df.select_dtypes(include=np.number)

corr_matrix = numerical_df.corr()

print(corr_matrix)
```

Output:

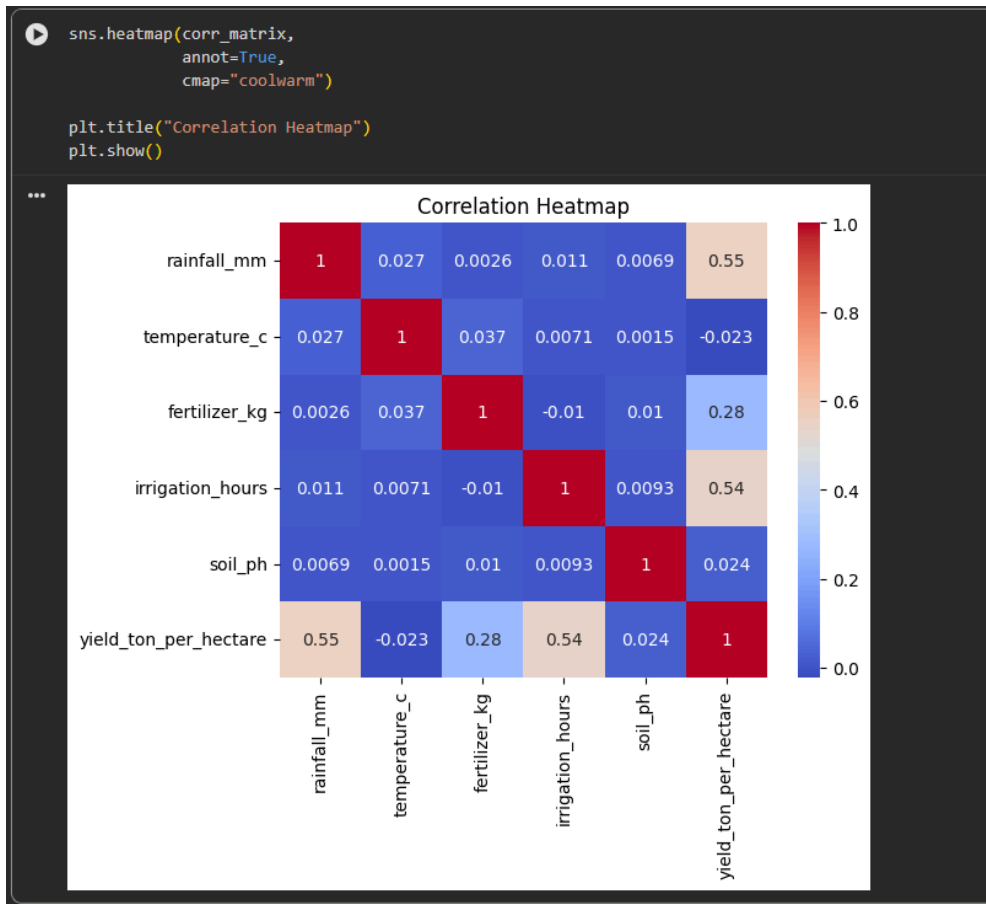


Code:

```
sns.heatmap(corr_matrix,
             annot=True,
             cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.show()
```

Output:



Code:

```
corr = corr_matrix["yield_ton_per_hectare"]
```

```
print(corr.sort_values(ascending=False))
```

Output:

```
corr = corr_matrix["yield_ton_per_hectare"]

print(corr.sort_values(ascending=False))
```

```
... yield_ton_per_hectare    1.000000
     rainfall_mm           0.553704
     irrigation_hours       0.542664
     fertilizer_kg          0.278043
     soil_ph                0.024412
     temperature_c         -0.022559
     Name: yield_ton_per_hectare, dtype: float64
```

Q10. Average Yield by Crop Type

Code:

```
crop_avg = df.groupby("crop_type") [
    "yield_ton_per_hectare"
].mean()
print(crop_avg)
```

Output:

```
crop_avg = df.groupby("crop_type") [
    "yield_ton_per_hectare"
].mean()

print(crop_avg)

... crop_type
Cotton    4.687299
Maize     4.897143
Rice      5.494744
Soybean   5.173431
Wheat     4.989472
Name: yield_ton_per_hectare, dtype: float64
```

Code:

```
soil_avg = df.groupby("soil_type") [
    "yield_ton_per_hectare"
].mean()

print(soil_avg)
```

Output:

```
soil_avg = df.groupby("soil_type") [
    "yield_ton_per_hectare"
].mean()

print(soil_avg)

... soil_type
Clay      5.134326
Loamy     5.366519
Sandy     4.588882
Name: yield_ton_per_hectare, dtype: float64
```

Code:

```
print(crop_avg.idxmax())
```

Output:

```
print(crop_avg.idxmax())

Rice
```

Code:

```
print(soil_avg.idxmax())
```

Output:

```
print(soil_avg.idxmax())

Loamy
```

Part C: Data Preparation

Q11. Feature Encoding

Code:

```
print(df.select_dtypes(include="object").columns)
```

Output:

```
print(df.select_dtypes(include="object").columns)
... Index(['crop_type', 'soil_type'], dtype='object')
```

Code:

```
encoded_df = pd.get_dummies(
    df,
    columns=["crop_type", "soil_type"],
    drop_first=True
)
```

```
print(encoded_df.head())
```

Output:

```
...    rainfall_mm  temperature_c  fertilizer_kg  irrigation_hours  soil_ph \
0         588.6         18.6         242.4         6.5         6.5
1         772.8         34.6         247.2         10.0         6.5
2         970.9         36.3         168.4         7.3         6.4
3         611.7         19.0         121.7         3.7         6.0
4         696.1         29.6         184.6         5.1         6.1

    yield_ton_per_hectare  crop_type_Maize  crop_type_Rice  crop_type_Soybean \
0              5.92             True      False      False      False
1              6.24             True      False      False      False
2              4.77             False     False     False      False
3              4.04             True      False     False      False
4              5.17             False     False     False      False

    crop_type_Wheat  soil_type_Loamy  soil_type_Sandy
0             False             True             False
1             False             False             True
2              True             False             True
3             False             True             False
4             False             False             False
```

Q12. Feature Selection

Code:

```
X = encoded_df.drop(
    "yield_ton_per_hectare",
    axis=1
)

y = encoded_df["yield_ton_per_hectare"]

print(X.head())
print(y.head())
```

Output:

```
...   rainfall_mm  temperature_c  fertilizer_kg  irrigation_hours  soil_ph  \
0         588.6          18.6         242.4           6.5         6.5
1         772.8          34.6         247.2          10.0         6.5
2         970.9          36.3         168.4           7.3         6.4
3         611.7          19.0         121.7           3.7         6.0
4         696.1          29.6         184.6           5.1         6.1

   crop_type_Maize  crop_type_Rice  crop_type_Soybean  crop_type_Wheat  \
0             True           False           False           False
1             True           False           False           False
2             False          False           False            True
3             True           False           False           False
4             False          False           False           False

   soil_type_Loamy  soil_type_Sandy
0             True           False
1             False            True
2             False            True
3             True            False
4             False            False
0         5.92
1         6.24
2         4.77
3         4.04
4         5.17
Name: yield_ton_per_hectare, dtype: float64
```

Part D: Machine Learning

Q13. Train-Test Split

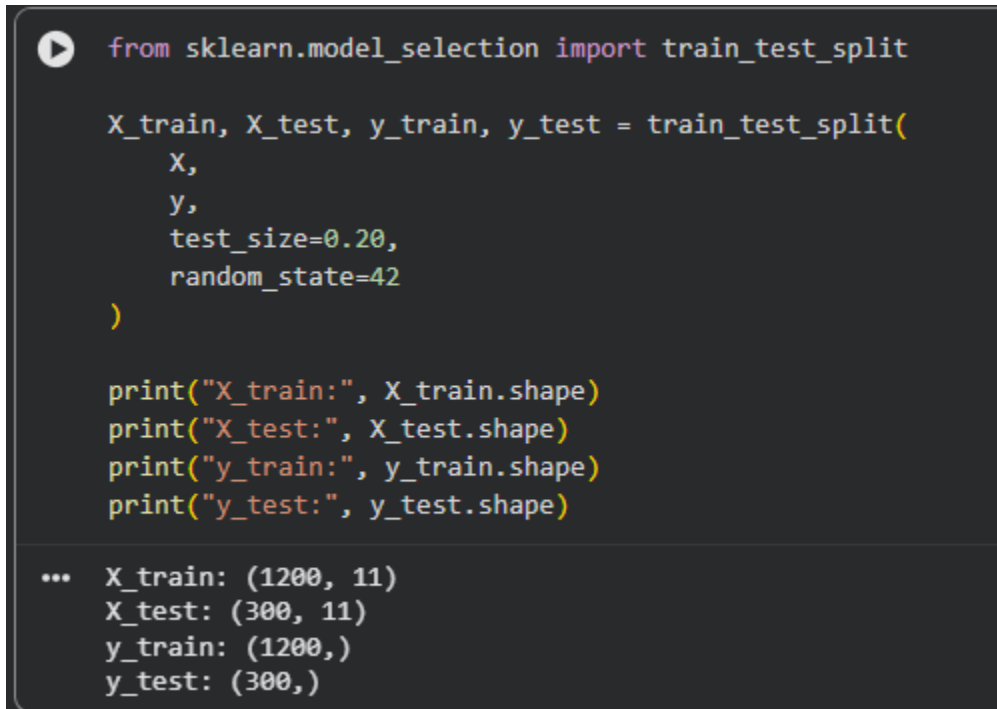
Code:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

Output:



```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

... X_train: (1200, 11)
X_test: (300, 11)
y_train: (1200,)
y_test: (300,)

Q14. Linear Regression Model

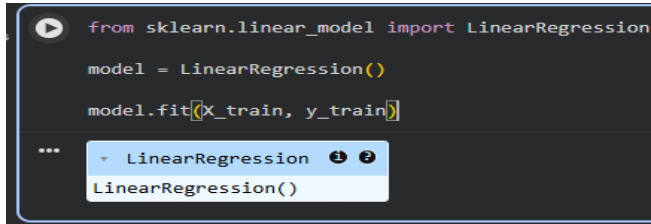
Code:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)
```

Output:



```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

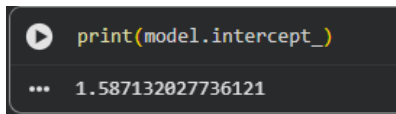
model.fit(X_train, y_train)
```

... LinearRegression LinearRegression()

Code:

```
print(model.intercept_)
```

Output:



```
print(model.intercept_)
```

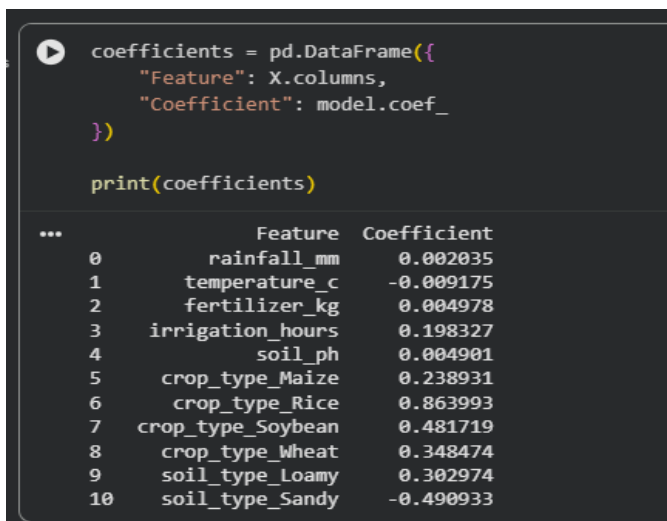
... 1.587132027736121

Code:

```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print(coefficients)
```

Output:



```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print(coefficients)
```

... Feature Coefficient

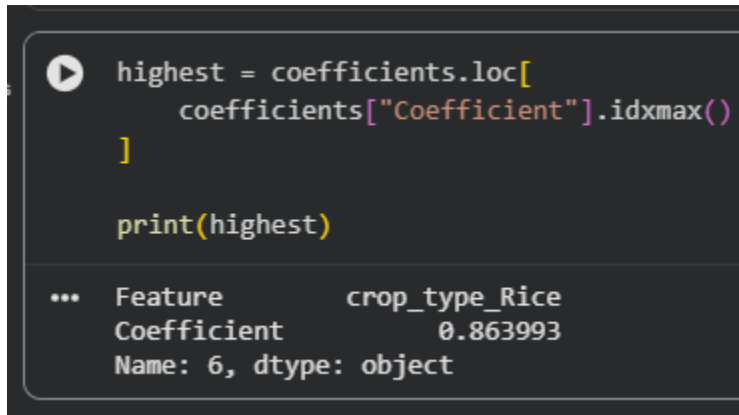
	Feature	Coefficient
0	rainfall_mm	0.002035
1	temperature_c	-0.009175
2	fertilizer_kg	0.004978
3	irrigation_hours	0.198327
4	soil_ph	0.004901
5	crop_type_Maize	0.238931
6	crop_type_Rice	0.863993
7	crop_type_Soybean	0.481719
8	crop_type_Wheat	0.348474
9	soil_type_Loamy	0.302974
10	soil_type_Sandy	-0.490933

Code:

```
highest = coefficients.loc[
    coefficients["Coefficient"].idxmax()
]

print(highest)
```

Output:



```
highest = coefficients.loc[
    coefficients["Coefficient"].idxmax()
]

print(highest)
```

...	Feature	crop_type_Rice
	Coefficient	0.863993
	Name:	6, dtype: object
